

Comparative Analysis of Sign Language Dataset

Ece Melisa Kocak

DATA 604: Data Representation and Modeling

Professor Wojciech Czaja

May 9, 2026

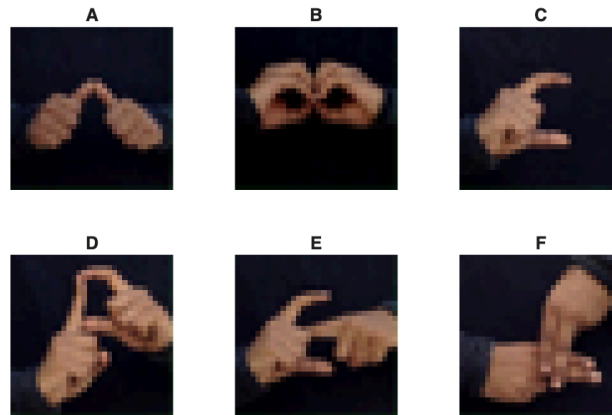
1 Introduction

This analysis is designed to answer the question: “Is data representation helping or hurting classification?”. This analysis will also outline how each representation method performed in terms of computational runtime, representation quality, and other relevant evaluation metrics. For this project, the representation methods include raw data, PCA (Principal Component Analysis), kPCA (kernel PCA) and Isomap. Additionally, kernel SVM was implemented for classification. Classification was evaluated in terms of accuracy, F1 score and confusion matrices. High-dimensional datasets are more complex to analyze because of their complicated structure. This analysis will provide insight into the tradeoffs and benefits of different various methods when applied to the same dataset. This project’s logic relies on an expectation that as more contextually-appropriate analysis methods are applied, from raw data, to PCA, to nonlinear techniques, the classification will improve in turn. If it is the case that optimized methods help classification, then these changes in technique should be reflected in the evaluation metrics. If it is not the case that representation helps, then there may be no relationship between method and output quality, or methods that are more theoretically optimized may actually inhibit the quality of the classification.

2 Data Source

For this analysis, I retrieved the Sign Language Detection Using Images dataset from Kaggle. This dataset has 35 distinct classes where each class has 1200 distinct examples. The dimensionality for each image is $128 \times 128 \times 3$. As a result, there are 49,152 features for each sample, and 42,000 elements in total in the dataset. The dataset includes distinct signs intended for communication, but its letters are not derived from a specific regional signed language. The data is labeled and includes the numbers one through nine along with all twenty-six letters in the English alphabet. In each class, the sign is represented by several images. Figure 1 reflects an example of each of the first 6 classes in the dataset. Each example is slightly different with changes to the background and lighting. This variety can improve the model’s robustness when encountering changes to the context of a sign’s presentation.

Figure 1: First 6 classes in the dataset



3 Data Preprocessing

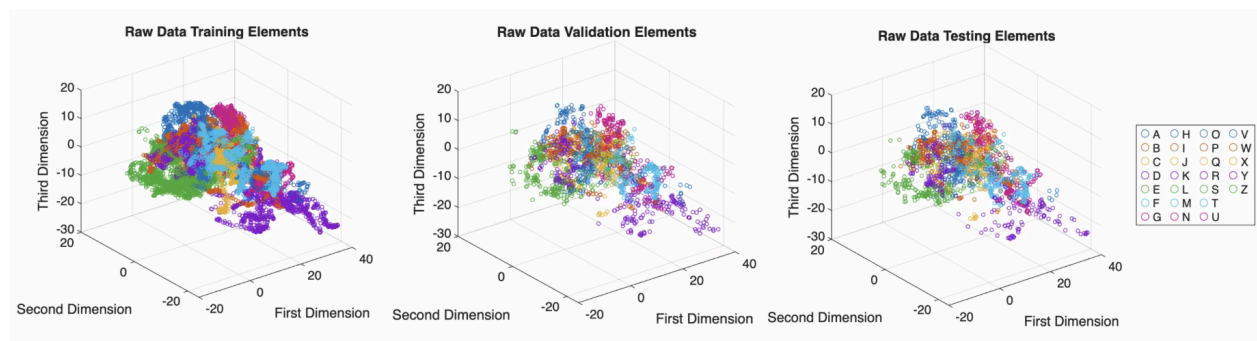
The first data processing step was to reduce the image size to minimize the computational cost and the risk of overfitting the model. With resizing, I minimized memory use and performed operations much faster. The new dimensionality for each image was $32 \times 32 \times 3$. After removing the classes for each digit, the number of remaining classes was 26. Image files were also converted into vectors which improved their fidelity. After limiting to 800 distinct examples in data preprocessing, a total of 20,800 images were used to evaluate the features of the raw data and to construct the PCA. The sample was reduced further prior to the kPCA and Isomap implementation. Additionally, I normalized the pixel values for future comparison and to prevent large values from dominating.

4 Representation Methods

4.1 Raw Data

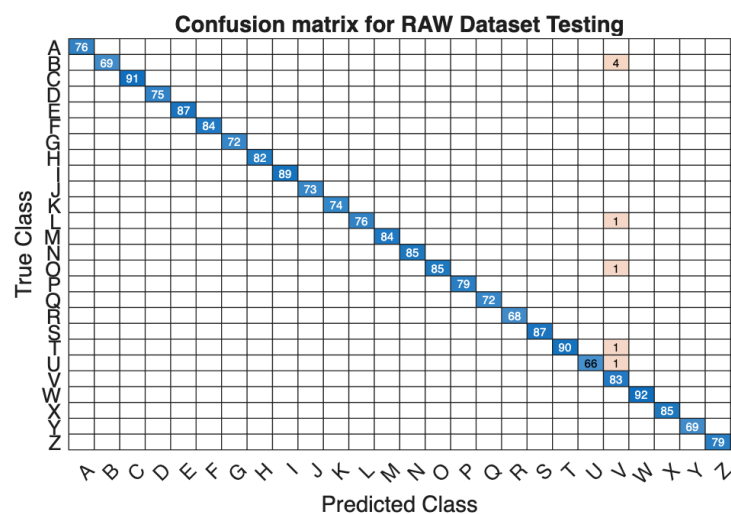
The raw data is presented below in a 3D plot with random projection. With random projection, I reduced the high dimensionality and computed the projected data. As a result, raw data took only 0.049266 seconds to compute. Figure 2 reflects the split of training (80%), validation and testing elements (10% each). By visualizing the split, I verified that my class distribution is consistent. There are many clusters overlapping and no separation of the boundaries visually for the classes. Therefore, further representation methods are needed as raw data visualization is insufficient for understanding the data.

Figure 2: Raw Data Split into Training, Validation and Testing Elements



Despite the overlapping clusters in the 3D plots of raw data, the classification performance was very good, achieving accuracy for testing data around 99%. This means that the Gaussian kernel SVM identified the separation of classes efficiently. Observing the confusion matrix in Figure 3, most of the classes are classified correctly with few misclassifications.

Figure 3: Confusion matrix for Raw Dataset Testing Elements



Overall, in the raw data visualization could be more distinct visually, but in reality the classes are separated well as the SVM classifier effectively identifies hand gesture shapes, patterns and various differences. The gestures in the data are consistent in shape, and this consistency likely helps the classification accuracy. This aligns with the nature of signed language in that these forms are meant for communication and mutual understanding. The language was designed with the goal of all signs being clearly separate from one another, and this design is directly affecting the quality of raw data classification.

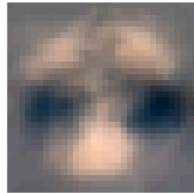
4.2 PCA

Principal Component Analysis (PCA) is an essential linear dimensionality reduction technique as it embeds “the data into a linear subspace of lower dimensionality” (Maaten, Postma, & Herik, 2008). PCA helps visualize the clustering and classification within a dataset, and it maintains effectiveness despite high dimensionality, noise, and complexity. This process makes it easier to find the directions of maximum variance.

While compressing the data, PCA also removes redundancies. The first and last principal components are displayed in Figure 4. The first principal component captures the most significant variation in the dataset and it successfully maintains the structure of letters. Information preservation, and in turn the relative importance of the variation, reduces between each component. Variance refers to the amount of difference across dimensions, and as the amount of information degrades, the variation degrades along with it. Because earlier components have more information, they also differ from one another with a greater degree of variance. On the other hand, the last principal component shows primarily noise in the bottom left. There is not a clear pattern, and the component only captures a small amount of variance.

Figure 4: First and Last Principal Component

The First Principal Component



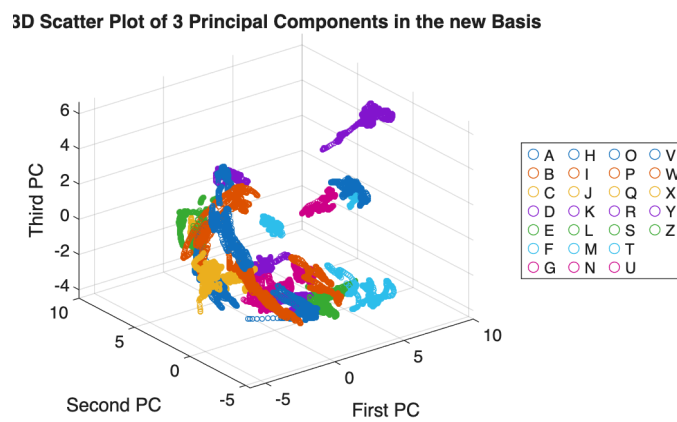
The Last Principal Component



For this analysis, PCA was calculated with MATLAB’s `pca` function rather than with the help of `drtoolbox` as the data is mean-centered prior to calculating PCA. The first 3 principal components were visualized as they capture most variance. The 3D projection of the first 3 principal components provides several key insights (Figure 5). First, each point portrays the individual image reduced to 3D. The visible clusters indicate that PCA successfully identified

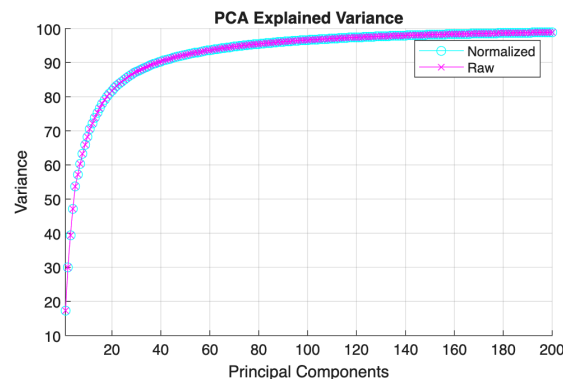
patterns in the hand gestures. Towards the center of the plot, there are various overlapping clusters, which suggests overlap in the structure of these gestures. The curved clusters could be due to the nonlinear relationships between the parameters of clustering and their meaning or symbolic relationship. PCA only preserves linear variance globally which is a limitation that, in theory, will be evaded by nonlinear methods such as kPCA and Isomap. The first principal component provides the most information as it efficiently captures the variance and differences between signs.

Figure 5: 3D Scatter Plot of 3 Principal Components in the New Basis



Normalization had little to no effect on PCA outputs. In terms of variance, the explained variance curve increased rapidly initially then slowed down (Figure 6).

Figure 6: PCA Explained Variance



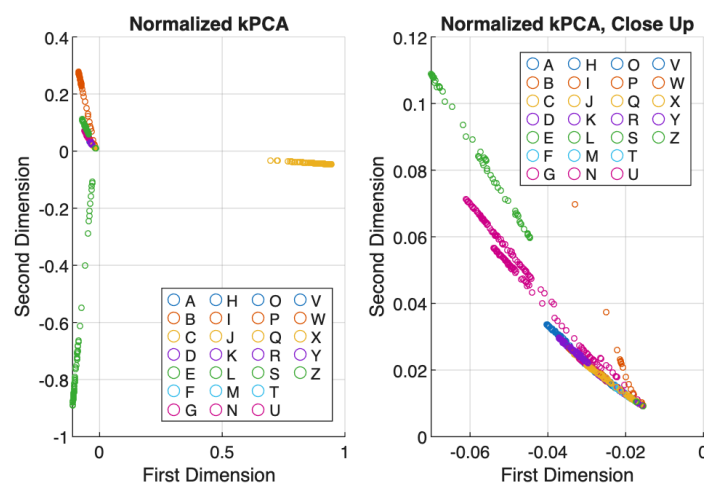
In this data, 95% of the variance can be explained by 74 principal components. Considering the diminishing variance captured by each component, the first 50 components were selected as they still capture a large amount of variance. With fewer features, the computational

cost is lower with 5.511085 seconds to process and the output has an outstanding classification performance. Overall, PCA performed much better than the raw data in terms of both visualization and across all evaluation metrics.

4.3 kPCA

Kernel Principal Component Analysis (kPCA) is optimal for nonlinear data as it can compute the eigenvectors of the kernel matrix. kPCA has been referred to as, “the reformulation of traditional linear PCA in a high-dimensional space that is constructed using a kernel function” (Maaten, Postma, & Herik, 2008). As opposed to PCA, kPCA captures the nonlinear structure and will portray the clusters more distinctly. The raw kPCA data presents more noise and tighter clusters than the conventional PCA data. With normalization, pixels were scaled and the slight changes in images no longer dominated the distance. Therefore, normalization had a tremendous effect on kPCA as the clusters of classes became more meaningful. The close up plot shows the clusters more clearly (Figure 7). It is important to note that kPCA has a higher computational cost due to the kernel matrix. As a result, the number of samples used were reduced to 100 per class, similar to lecture code. By doing this, kPCA took 3.592986 seconds to run, which is much faster and more stable compared to running on a larger number of samples.

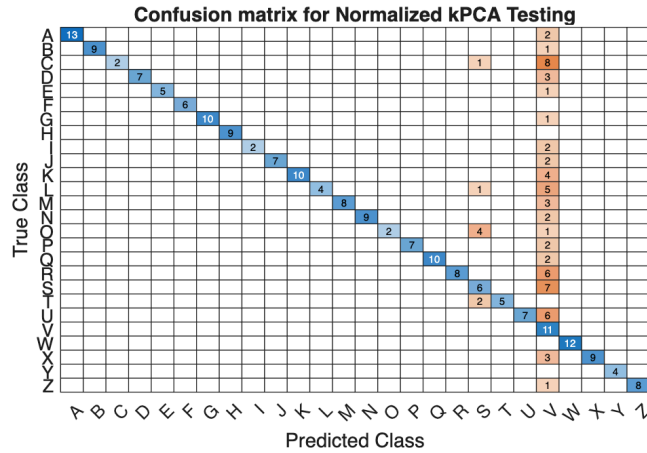
Figure 7: Normalized kPCA



Using the SVM classifier for the normalized kPCA showed much lower results compared to the previous representation methods. For testing data, accuracy was 73.0769% and F1 score was 79.0422%. The low results are shown in the confusion matrix in Figure 8. The confusion

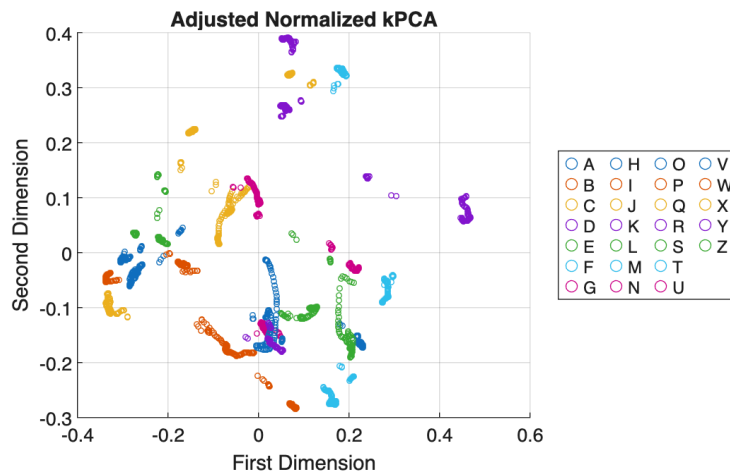
matrix reflects that the normalized kPCA had various misclassifications which resulted in classifying worse than raw data or PCA. Although the visual representation of normalized kPCA was superior with distinct clusters for each class, these gains in representation were joined with poorer classification. This suggests that the relationship between representation and classification is not linear.

Figure 8: Confusion matrix for Normalized kPCA Testing Elements



To improve the classification, I hypertuned the parameters under which kPCA was run. I expected that kPCA would classify more accurately as it preserves the nonlinear relationships in the data, so I wanted to adjust the parameters to see if this could be manifested. The normalized kPCA was adjusted to the Gaussian kernel with sigma = 15 and tested with 40 features (Figure 9). The Gaussian Kernel was picked as it preserved the curved manifolds more clearly. With hypertuning and observation of the various parameters, the clearest separation was seen using sigma = 15.

Figure 9: Normalized kPCA with Gaussian Kernel, when sigma is 15

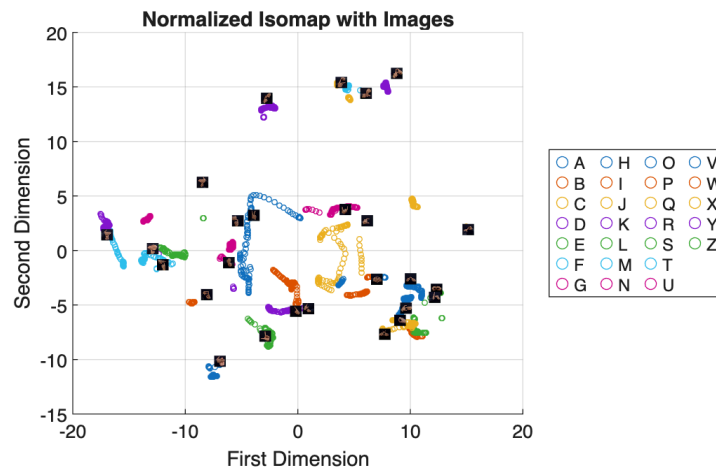


Comparing Figure 7 and 9, there are fewer overlapping clusters in Figure 9. kPCA ran on data that was reduced in comparison to previous representation methods, but, once adjusted and normalized, it performed equally well. The normalized kPCA with Gaussian kernel, when sigma is 15, performed at ceiling for all of the evaluation metrics. Overall, kPCA provides a much clearer representation as it is able to capture the nonlinear relationship of the data that PCA had previously struggled to preserve. This improved representation comes at the expense of additional data treatment and hyperparameter manipulation.

4.4 Isomap

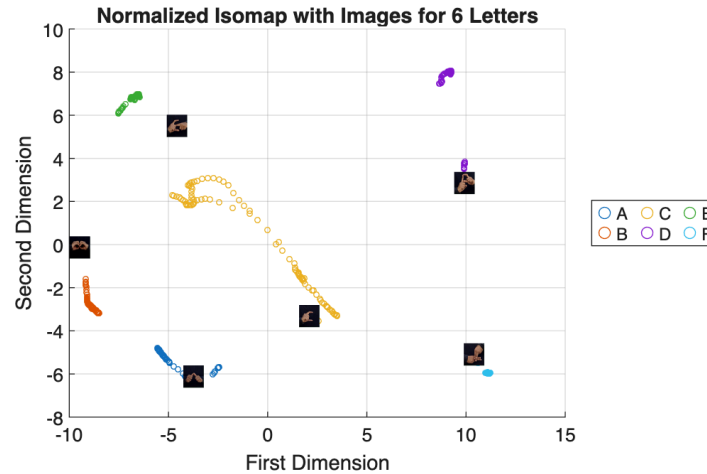
Isomap is a convex and full spectral representation method that includes geodesic distance. With the dataset, the geodesic distance represents the k nearest neighbor as it portrays “the distance between two points measured over the manifold” (Maaten, Postma, & Herik, 2008). I chose isomap because of its implementation in head pose estimation, which, similarly to my dataset, relies on modeling slight changes in lighting and shadows against the body. With Isomap, the goal was to detect minor linguistically salient changes and classify the hand gestures more precisely using those features. Before this analysis, I identified the limitations of Isomap which are short-circuiting, topological instability, and noise. The number of classes were again reduced to 100 in order to lower the computational complexity. By doing this, Isomap took 114.203247 seconds to run. By reducing the input, I was able to focus on connected neighbors. More specifically, focusing on 300 neighbors, I was able to preserve the manifold structure both locally and globally. Figure 10 displays a 2D embedding of normalized isomap with images.

Figure 10: Normalized Isomap with Images



Isomap produces similar output to adjusted normalized kPCA. However, with Isomap the trajectories are more interpretable while retaining the curved manifold. Similar signs are clustered together, whereas opposite signs are clearly separate. Isomap’s classification was also high and it had very few misclassifications. To see Isomap close up, I decided to explore it with the first 6 letters of the alphabet which are displayed in Figure 11.

Figure 11: Normalized Isomap with Images for First 6 Letters



There are a number of interesting things to note in figure 11. First, the trajectory of “C” (yellow) approaches very nearly the space for the letter “E” (green). This is likely due to the similar base shape of the two gestures. They both involve one hand forming a ‘C’ shape, but the “E” adds a finger from the other hand in the middle. The underlying similarity of these gestures is likely responsible for the near-convergence of the clusters. Another interesting detail is the distance between the clusters for “B” (orange) and “D” (purple). Similarly to “C” and “E”, “B” and “D” share a primary gesture in the formation of a round hand shape. Here however, the clusters are clearly far apart. This could be due to the different number of hands involved in these signs or the different shape and number of openings produced by the hands. In any case, the similarity between these gestures is clearly well captured by Isomap.

5 Classification

RBF Kernel SVM was used to classify the complex, nonlinear sign language gestures. SVM handles big data classification more efficiently while capturing the nonlinear structure. Although SVM are binary by definition, since I worked with 26 classes in the dataset, I used

MATLAB's `fitcecoc` function in order to extend it to multiclass classification. I also observed the first five class predictions with their probabilities, after each classification of a representation method. This was done to track the classifier's confidence. I applied the evaluation metrics using SVM. Due to skepticism about the high classification accuracy, I applied cross validation in order to test while running 3 different SVM models.

5.2 Training and Validation

As stated previously, for training the SVM classifier, 80 percent of the dataset was used to fit model parameters. For validation and testing sets, 10% each were used. The split ratio was preserved with all of the methods. The validation set is for tuning hyperparameters and the test set is for final evaluation on elements that were not for hypertuning. As a result of the separation of the data, I was able to reduce overfitting as well as perform a consistent evaluation across the different representation methods. With SVM, I used evaluation metrics on validation and testing data, which will be elaborated on in the coming sections.

5.3 Evaluation Metrics

To evaluate model performance, I used multiple classification metrics. For each model, I computed the accuracy, F1 score and confusion matrix. Accuracy reflects the overall percentage of correct predictions. The F1 score combines precision and recall into a single metric which reflects the model's performance. Lastly, the confusion matrix shows the distribution of true positives, false positives, true negatives and false negatives. To obtain these evaluation metrics, SVM was trained on the training dataset and then evaluated with the validation and testing datasets. As mentioned previously, 3 fold cross validation was applied using `KFold` with accuracy and F1 score.

6 Results

Representation Methods	3-fold CV: Accuracy	3-fold CV: F1 score	Runtime (seconds)	Key Parameters
Raw Data	97.9231%	98.1238%	0.049266	N/A
PCA	99.9459%	99.9464%	5.511085	50 Principal

				Components
Normalized kPCA	71.1538%	77.5225%	3.592986	Gaussian Kernel, $\sigma=15$
Normalized & Adjusted kPCA	100%	100%		
Normalized Isomap	99.0769%	99.0768%	114.203247	300 neighbors

In theory, reducing dimensionality should lower the computational cost, potentially at the expense of classification. For this dataset, a different trend emerged. From the start, raw data was already highly accurate despite poor visual representation. This is likely due to the intentional distinctness and clarity of the signs. On the other hand, PCA not only performed better than raw data with classification but also was able to produce a better visualization. The nonlinear methods, kPCA and Isomap, were able to visually separate each class using the nonlinear relationship between signs. These two methods involved higher computational complexity and the data had to be reduced for efficiency and stability. To answer the question, “Is data representation helping or hurting classification?”, this analysis gives a nuanced affirmative. While the quality of classification did reach ceiling with PCA and normalized and adjusted kPCA, the relative benefits of these techniques may have been outweighed by their additional costs for this particular dataset. I suspect that if the models were trained using nonlinear data that had more overlap than the 26 unique signs, it is likely that raw data would reduce in utility while the nonlinear methods became more valuable. In this analysis, the benefits of more complicated methods were relatively minimal, but the trend of improved performance would likely be clearer on different data.

8 References

Maaten, L.V., Postma, E.O., & Herik, J.V. (2008). *Dimensionality Reduction: A Comparative Review*.

Data Source:

<https://www.kaggle.com/datasets/harshvardhan21/sign-language-detection-using-images>